

四、优化RAG应用提升回答准确度

--楼兰

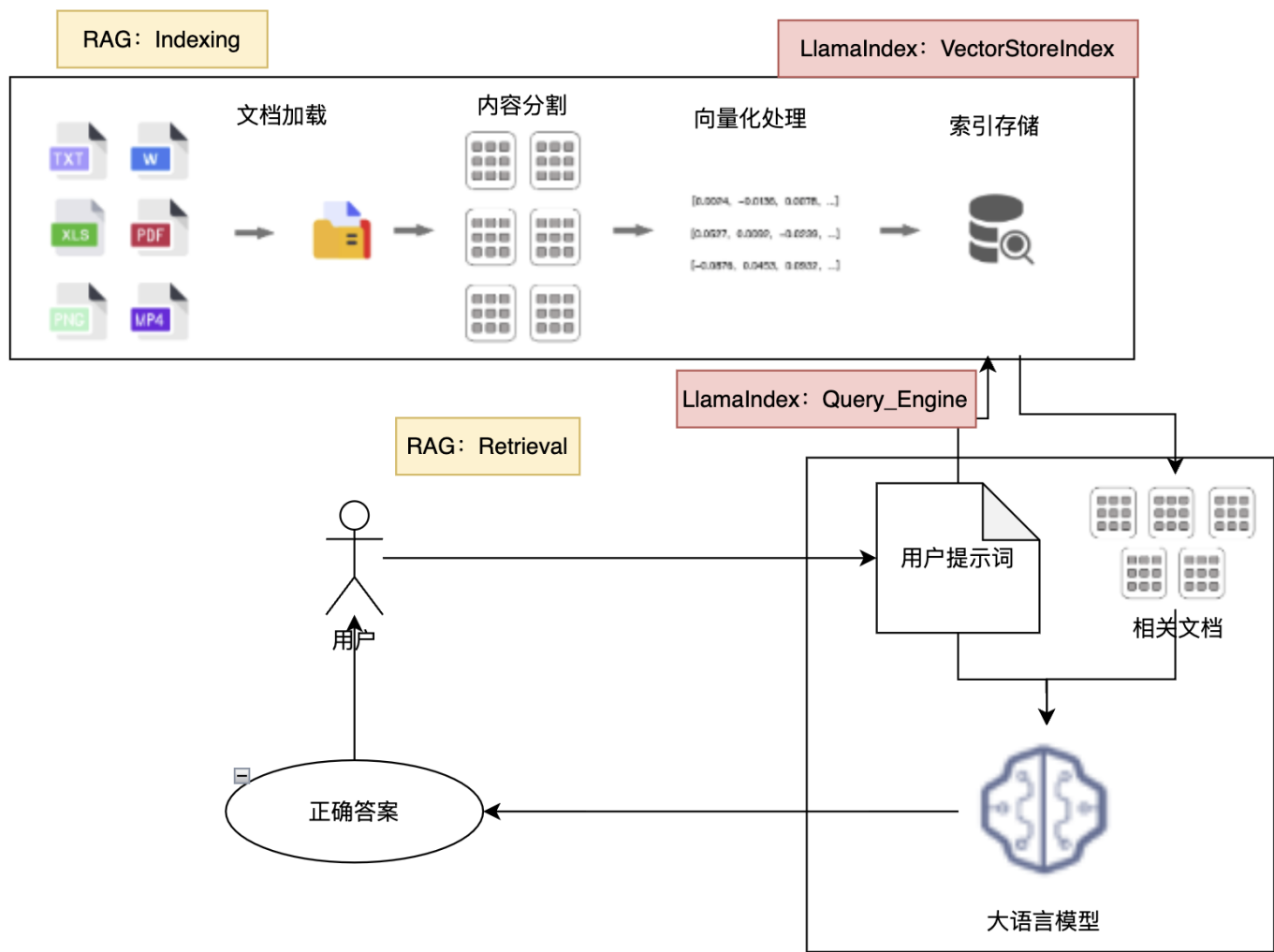
章节目标：整理RAG应用提升回答准确度的思路

在上一章节通过构建评测体系，已经能够对RAG应用的各个阶段效果进行评测，接下来自然就是参考这些评测结果，对RAG应用的各个步骤进行针对性的调优了。

因为RAG应用可能存在的问题非常多，无法通过案例一一展示。所以这一章节重在梳理和经验。

一、RAG应用回答准确性问题分析

回顾下RAG的整体路程如下：



然后，通过上一章节的评测，已经发现，我们之前的RAG应用回答问题效果不好的根本原因就在于检索出来的相关信息太少，没有包含真实的条目。所以，我们可以针对性的增加召回的信息条目的数量。

```
import logging
logging.basicConfig(level=logging.ERROR)
```

```

from utils import llm, embed_model
from llama_index.core import Settings, SimpleDirectoryReader, VectorStoreIndex
import logging
logging.basicConfig(level=logging.ERROR)

Settings.llm=llm
Settings.embed_model=embed_model

documents = SimpleDirectoryReader(input_dir="./docs").load_data()
index = VectorStoreIndex.from_documents(documents)
# 构建引擎，一次检索出5个文档切片，默认为2
query_engine = index.as_query_engine(similarity_top_k=4)
response = query_engine.query("王芳是哪个部门的? ")
response.response

```

可以看到，当我们把query_engine中检索相关文档片段数增加到5时，就能够检索出王芳相关的部门信息了。这就是针对之前评测结果的一种解决方案。

不过，单纯增加召回的切片数量可能并不是一个好办法。想想看，如果用这种方法解决所有问题，那么不如召回整个知识库，这样不会遗留任何信息。但是显然，这不仅会超出大模型的输入长度限制，而且过多的无关信息还会降低大模型回答的效率和准确性。

所以我们还可以在这个基础上，继续做一些优化。优化之前，我们对当前的RAG应用进行一次评估。

```

from langchain_community.embeddings import DashScopeEmbeddings
from datasets import Dataset
from ragas import evaluate
from ragas.metrics import answer_correctness, context_recall, context_precision
from config.load_key import load_key
# 定义评估函数
def evaluate_result(question, response, ground_truth):
    # 获取回答内容
    if hasattr(response, 'response_txt'):
        answer = response.response_txt
    else:
        answer = str(response)
    # 获取检索到的上下文
    context = [source_node.get_content() for source_node in response.source_nodes]

    # 构造评估数据集
    data_samples = {
        'question': [question],
        'answer': [answer],
        'ground_truth': [ground_truth],
        'contexts': [context],
    }
    dataset = Dataset.from_dict(data_samples)

    # 使用Ragas进行评估
    score = evaluate(
        dataset = dataset,
        metrics=[answer_correctness, context_recall, context_precision],

```

```
llm=llm,
embeddings=DashScopeEmbeddings(model="text-embedding-
v3",dashscope_api_key=load_key('BAILIAN_API_KEY'))
)
return score.to_pandas()
```

```
question = "王芳是哪个部门的?"
ground_truth = "王芳是教研部的教研专员"
evaluate_result(question, response, ground_truth)
```

可以看到，当前评测的context_precision只有0.25。

接下来，我们可以尝试下重新调整文档索引。将不太好拆解的PDF格式转换成结构更加规整的MarkDown格式来试试。

```
new_documents = SimpleDirectoryReader(input_dir="./markdown").load_data()
new_index = VectorStoreIndex.from_documents(new_documents)
# 构建索引
new_query_engine = new_index.as_query_engine(similarity_top_k=5)
new_response = new_query_engine.query("王芳是哪个部门的? ")
new_response.response
```

```
evaluate_result(question, new_response, ground_truth)
```

可以看到，这次的context_precision提升到了0.3333。检索信息的效率提高了。

二、RAG应用各个环节改进策略

之前的优化措施，还只是简单的修改了知识库的文件格式，就已经能够让RAG应用的回答效率得到提高。未来我们可能需要面临的是更复杂的场景，所以接下来有必要思考一下，在RAG的整个工作流程当中，有哪些重要的优化步骤，这样可以方便找到新的改进点。

1、文档准备阶段

对RAG应用来说，文档的质量可以说直接决定了整个RAG应用的质量。因此，我们需要针对文档进行持续的优化。比如引入专家验证，提升文档质量。跟踪用户反馈，不断调整文档内容。查漏补缺，同时定期剔除无关的或过时的内容，都是必要的措施。

2、文档解析与切片阶段

在使用LlamaIndex构建RAG应用时，他是会自动解析文档内容并进行切片的。这对于大部分的场景是足够的。但是，实际工作中，当文档变得更复杂时，对文档进行合理的解析和切片就变得非常重要了。

比如，在LlamaIndex中，通常使用SimpleDirectoryReader加载本地文件，实际上，LlamaIndex同样提供了一些扩展插件，可以加载Notion、Slack、Discord、Google Docs等外部的文档。具体可以参见：https://docs.llamaindex.ai/en/stable/module_guides/loading/connector/modules/

当文档多了之后，如果文档来源不统一，文档形式又五花八门，这是不利于统一进行解析的。我们上面就演示了把PDF格式的文档转换成为结构规整的Markdown格式，这样是可以提高RAG应用的效率的。对于docx\xlsx等格式的文档，如果结果过于复杂，把他们统一转换成为Markdown格式也是一个不错的选择。在做格式转换时，有些比较麻烦的问题，也可以借助大模型来进行深度调整。比如将转换的Markdown文本用大模型进行润色、修正目录层级、补充缺失信息等。

当文档被正确加载成Document后，接下来LlamaIndex也会完成对Document的切片工作，将Document切分成为多个相关联的Node。LlamaIndex中也提供了非常多的文档切分工具可以作为参考，当然，如果这些工具不够用时，也可以自己实现。

- TokenTextSplitter：按照Token进行拆分。比较适合对Token数量有严格要求的场景。比如使用上下文长度比较小的模型时。
- SentenceSplitter：这是LlamaIndex默认的切片策略，切片时会尽量保持句子的完整性。
- SentenceWindowNodeParser：基于句子进行切分。不过在切分时，每个Node会包含周围的句子作为上下文窗口，这样可以更好的保持句子的完整性。
- SemanticSplitterNodeParser：根据语义相关自适应进行切片。
- MarkdownNodeParser：基于Markdown格式进行切分。LlamaIndex提供的实现会根据Markdown文档的标题层级进行智能切分。

3、文本向量化与存储阶段

文档切片后，我们就需要对其建立索引，以便后续检查。最常见的方案就是使用EmbeddingModel向量化模型将切片转成向量，然后存储到向量数据库当中。

首先，你要清楚，向量本质上是把不精确的自然语言转成计算能够理解的精确的数字化的表达。转换出来的向量，主要包含的是自然语言的语义相似度信息。

```
from sklearn.metrics.pairwise import cosine_similarity
from llama_index.embeddings.dashscope import (
    DashScopeEmbedding,
    DashScopeTextEmbeddingModels,
    DashScopeTextEmbeddingType,
)
from config.load_key import load_key
import numpy as np
text1 = "我喜欢吃苹果"
text2 = "苹果是我最爱吃的水果"
text3 = "今天天气不错"

embedder = DashScopeEmbedding(
    model_name=DashScopeTextEmbeddingModels.TEXT_EMBEDDING_V3,
    text_type=DashScopeTextEmbeddingType.TEXT_TYPE_DOCUMENT,
    api_key=load_key("BAILIAN_API_KEY"),
)

vector1 = np.array(embedder.get_text_embedding(text1)).reshape(1, -1)
vector2 = np.array(embedder.get_text_embedding(text2)).reshape(1, -1)
vector3 = np.array(embedder.get_text_embedding(text3)).reshape(1, -1)
```

```
# 计算余弦相似度
similarity12 = cosine_similarity(vector1, vector2)[0][0]
similarity13 = cosine_similarity(vector1, vector3)[0][0]
print(f"\n\"{text1}\" 与 \"{text2}\" 相似度: {similarity12:.4f}")
print(f"\n\"{text1}\" 与 \"{text3}\" 相似度: {similarity13:.4f}")
```

这里需要注意的是，向量的本质是自然语言，而自然语言其实是一种非常不精确的表达方式。不同的人对语言的理解是不一样的，同样，不同的Embedding向量化模型对相同的文字计算，得到的向量也可能是完全不同的。例如上面的案例，可以分别使用阿里云提供的text-embedding-v2和text-embedding-v3模型，分别进行计算，得到的结果也是完全不同的。

不过，一个通常的经验是，越新的Embedding模型，其表现通常越好。

然后，如何选择向量数据库？

向量数据库，通常要做的事情就两件，一是把向量存储起来，二是根据向量进行相似度检索。而实现这两个功能，其实并不一定需要额外的数据库产品。LlamaIndex内置的向量存储就是使用内存完成。这样的优点是快速上手，适合开发测试。但是缺点也很明显，数据量受限于内存的大小。

当数据量增大时，可以使用开源的向量数据库，如Milvus、Qdrant等。这些数据库提供了数据持久化和高速检索的能力。很多传统的数据存储产品，也可以作为向量数据库使用。比如Redis、Elasticsearch等。这些本地数据库的优势是功能完整、可控性强。但是缺点是需要自行部署维护。

这时，也可以选择一些云服务提供的向量存储能力。例如阿里云上提供过的各种数据服务。像Redis也提供了Redis Cloud云上服务，可以直接使用。

这些不同的向量数据库虽然具体的实现方式是天差地别的，但是他们的核心功能其实是差不多的。选择不同的向量数据库，通常不会对RAG应用的准确性产生太大的影响。

4、检索召回阶段

检索阶段会遇到的主要问题是，很难从众多的文档切片中，找到和用户问题最相关、且包含正确答案信息的片段。所以这个阶段的优化过程，更多需要考虑如何提升检索的性能。

要优化检索召回的效果，通常要分为两个不同的思路：

- 在执行检索前，需要更多关注用户的问题。减少用户问题中描述不完整、甚至有歧义的地方，更好的还原用户的真实意图。
- 在执行检索后，可能会出现很多和用户问题无关的信息。这时需要想办法减少无关信息，避免干扰下一步生成答案的质量。

常见的策略有以下几种：

1. 用户提问前，对问题进行扩展

例如，将“找王芳”主动改写成“王芳是哪个部门的？他的联系方式、职责范围、工作目标是什么”，这样能让目标更清晰。

例如，在用户询问“工作注意事项有哪些”时，主动根据用户的个人信息进行改写，扩展成“项目经理的工作注意事项有哪些”。

对问题进行适当扩写，可以极大的提升检索的准确度。当然，在具体实现过程中，扩展的规则会有很多，你甚至可以引入大模型，让大模型来帮助进行问题扩写。

2. 将用户的单一查询，改写成多步问题

LlamaIndex中就提供了两个强大的工具来实现这个功能：

- StepDecomposeQueryTransform: 这个工具可以帮你把一个复杂问题分解成多个子问题。比如对于“王芳是哪个部门的”，他就可以分解成“公司里有几个叫王芳的员工？”和“这些王芳分别在哪些部门？”两个问题，这样可以更全面地获取所有王芳的信息
- MultiStepQueryEngine: 这个查询引擎会按顺序处理这些子问题。它会先获取公司所有王芳的信息，然后依次查询每个王芳的部门信息，最终答案聚合成一个完整回应。

把大问题拆解成小问题往往更容易得到准确的答案。不过需要注意的是，LlamaIndex的实现过程是通过多次调用大语言模型完成的，所以会消耗更多的Token。另外，也会带来过程的不确定性。

```
from IPython.display import display
from llama_index.core.indices.query.query_transform.base import (
    StepDecomposeQueryTransform,
)
step_decompose_transform = StepDecomposeQueryTransform(verbose=True)
# set Logging to DEBUG for more detailed outputs
from llama_index.core.query_engine import MultiStepQueryEngine
query_engine = index.as_query_engine(streaming=True, similarity_top_k=5)
multi_step_query_engine = MultiStepQueryEngine(
    query_engine=query_engine,
    query_transform=step_decompose_transform,
    index_summary="公司人员信息"
)
question = "查找王芳的信息"
ground_truth = "王芳是教研部的教研专员"

print(f"? 用户问题: {question}\n")
print("AI正在进行多步查询...")
multi_step_response = multi_step_query_engine.query(question)
print("\n 参考依据:")
print("-" * 40)
for i, node in enumerate(multi_step_response.source_nodes, 1):
    print(f"\n文档片段 {i}:")
    print("-" * 30)
    print(node.text)

# 评估结果
print("\n 多步查询评估结果:")
print("-" * 40)

print("\n AI回答:", multi_step_response)
```

3. 使用假设文档HyDE进行增强

这种方法的基础思路是先让大模型基于问题编写一个“假象的答案文档”，然后用这个假象的文档来检索真实文档。最后用检索到的真实文档来生成实际答案。

```
from llama_index.core.indices.query.query_transform.base import (
    HyDEQueryTransform,
```

```

)
from llama_index.core.query_engine import TransformQueryEngine
# run query with HyDE query transform
hyde = HyDEQueryTransform(include_original=True)
query_engine = index.as_query_engine(streaming=True, similarity_top_k=5)
query_engine = TransformQueryEngine(query_engine, query_transform=hyde)

print(f" 用户问题: {question}\n")
print("  AI正在通过 HyDE 分析...")
streaming_response = query_engine.query(question)

print("\n AI回答:")
print("-" * 40)
streaming_response.print_response_stream()

# 显示参考文档
print("\n 参考依据:")
print("-" * 40)
for i, node in enumerate(streaming_response.source_nodes, 1):
    print(f"\n文档片段 {i}:")
    print("-" * 30)
    print(node.text)

print("\n AI回答:", streaming_response)

```

这个过程中，系统是如何生成“假想文档”的呢？下面可以看看AI实际生成了什么内容。

```

query_bundle = hyde(question)
hyde_doc = query_bundle.embedding_strs[0]
print(f"  AI生成的假想文档:\n{hyde_doc}\n")

```

可以看到，LlamaIndex在这个过程中用AI完全编造了一份王芳的信息，这样就可以按照这份假想文档的风格来检索真实员工的信息，形成更规范的回答。

5、重排序Rank

想象一下，如果是一个人来回答RAG最终的问题，他会希望参考信息怎么组织呢？是不是希望参考信息尽量简单，不要包含多余的信息，而且重要参考信息排在最前面。这样才能更方便的找到答案。这个事情，其实也可以让大语言模型来处理。例如阿里云百炼上提供了文本重排序模型，我们可以直接调用这个模型来对检索出的文本做一下处理。

深度文本重排序 gte-rerank-v2 排序模型 供应商：通义

模型介绍

gte-rerank-v2是通义实验室研发的多语言文本统一排序模型，面向全球多个主流语种，提供高水平的文本排序服务。通常用于语义检索、RAG等场景，可以简单、有效地提升文本检索的效果。给定查询 (Query) 和一系列候选文本 (documents)，模型会根据与查询的语义相关性从高到低对候选文本进行排序。

安装依赖

```
!pip install llama-index-postprocessor-dashscope-rerank-custom
```

```
from llama_index.postprocessor.dashscope_rerank import DashScopeRerank
from llama_index.core.postprocessor import SimilarityPostprocessor
rerank_query_engine = index.as_query_engine(
    # 先设置一个较大的召回切片数量
    similarity_top_k=20,
    streaming=True,
    node_postprocessors=[
        # 在rerank 模型中选择你最终想召回的切片个数，重排模型选择通义实验室的gte-rerank模型
        DashScopeRerank(top_n=3, model="gte-rerank", api_key=load_key("BAILIAN_API_KEY")),
        # 设置一个相似度阈值，低于该阈值的切片会被过滤掉
        SimilarityPostprocessor(similarity_cutoff=0.2)
    ]
)
reranked_response = rerank_query_engine.query(question)
# 显示参考文档
print("\n 参考依据:")
print("-" * 40)
for i, node in enumerate(streaming_response.source_nodes, 1):
    print(f"\n文档片段 {i}:")
    print("-" * 30)
    print(node.text)
print("\n AI回答:", reranked_response)
```

6、生成答案阶段

现在，大模型会根据你的问题和检索召回的内容，生成最终答案。这个过程更多的是由大模型进行处理，考验的是大模型的理解与总结能力。不过，你也可以从以下几个方面着手优化。

1. 选择合适的大模型

通常在这个阶段，大模型只需要做简单的信息总结，所以可以选择一些参数量比较小的模型就可以了。但是，如果你构建的RAG应用面向一些非通用领域，比如法律领域，那还是建议使用面向特定领域训练或者微调的模型。比如阿里云提供的通义法睿。

2. 充分优化提示词模板

最终召回的信息和用户的问题，是需要整理成一个完整的提示词提供给大语言模型的。LlamaIndex提供了默认的模板，将这些信息进行整合。但是，如果你希望大语言模型更理解你的要求，也可以自行修改模板。例如将默认的英文模板修改成中文。或者使用LlamaIndex的refine模式，让大模型对输出结果再做一次整理。

3. 调整大模型的参数

可以根据具体应用场景，适当调整大语言模型的参数。例如，如果你希望查询实时性的内容，可以适当降低temperature或top_p的值。如果是查询创造性的内容，可以适当增加他们的值。如果希望大模型在回答问题时不要总是用重复的句子，可以适当调高presence_penalty值。如果希望限制字数，控制成本或减少响应时间，可以适当降低max_tokens的值等。

4. 调优大模型

如果试了很多方法后，仍然不及预期，或者希望更进一步提升效果，这时也可以尝试面向当前的业务场景单独微调一个新的模型。当然，这样难度比较高，成本通常也比较大，可以适当关注。

三、总结

有了LlamaIndex框架的支持，RAG应用的实现过程会简单很多，但是如果你对RAG应用的效果有更高的要求，那么持续进行优化将是一个必不可少的过程。

当然，RAG的优化手段远不止这里分享的这些，业内对于RAG的研究和探索也从来没有停止过。这是一个需要持续关注的领域。

最后，对于LlamaIndex框架，其实他也是一个非常开放的框架。他的业务功能其实也不止构建RAG应用这么简单。包括处理多模态模型、构建Agent智能体以及多智能体协作系统等、LlamaIndex也提供个支持。另外，LlamaIndex也不是一个独立的框架，实际上通过他的良好设计，可以很轻松的将其他业务框架如LangChain等集成进来。所以，在LlamaIndex框架中，我们看到的不光是一个应用框架，而更多的是业内关于如何驯服AI大模型的行业经验。而这些经验，或许才是真正的Index。

